

Course Name: Advanced System Analysis and Software Design

Course Code: ENSF 480

Assignment Number: Lab 3

Lab Section: B01

Completed by:

Aayush Karthikeyan (UCID# 30189743)

Sarvesh Vettrivelan (UCID# 30242015)

Submission Date: 28/09/2026

Exercise-A

```
/*
 * File Name: main.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 28, 2026
 */
```

```
#include "graphicsWorld.h"
```

```
int main() {
    GraphicsWorld graphic;
    graphic.run();
    return 0;
}
```

```
/*
 * File Name: graphicsWorld.h
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 28, 2026
 */
```

```
#ifndef GRAPHICSWORLD_H
#define GRAPHICSWORLD_H
```

```
class GraphicsWorld {
public:
    void run();
};
```

```
#endif
```

```
/*
 * File Name: graphicsWorld.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 28, 2026
 */
```

```
#include "graphicsWorld.h"
#include "point.h"
#include "square.h"
#include "rectangle.h"
#include "circle.h"
```

```

#include "curveCut.h"
#include <iostream>
using namespace std;

void GraphicsWorld::run(){
#ifdef 1// Change 0 to 1 to test Point
    Point m(6, 8);
    Point n(6,8);
    n.setX(9);cout << "\nExpected to display the distance between m and n is: 3";
    cout << "\nThe distance between m and n is: " << m.distance(n);
    cout << "\nExpected second version of the distance function also print: 3";
    cout << "\nThe distance between m and n is again: "
        << Point::distance(m, n);
#endif // end of block to test Point

#ifdef 1// Change 0 to 1 to test Square
    cout << "\nTesting Functions in class Square:" <<endl;
    Square s(5, 7, 12, "SQUARE - S");
    s.display();
#endif // end of block to test Square

#ifdef 1// Change 0 to 1 to test Rectangle
    cout << "\nTesting Functions in class Rectangle:";
    Rectangle a(5, 7, 12, 15, "RECTANGLE A");
    a.display();
    Rectangle b(16, 7, 8, 9, "RECTANGLE B");
    b.display();
    double d = a.distance(b);
    cout <<"Distance between square a, and b is: " << d << endl;

    Rectangle rec1 = a;
    rec1.display();

    cout << "\nTesting assignment operator in class Rectangle:" <<endl;
    Rectangle rec2(3, 4, 11, 7, "RECTANGLE rec2");
    rec2.display();
    rec2 = a;

    a.set_side_b(200);
    a.set_side_a(100);

    cout << "\nExpected to display the following values for objec rec2: " << endl;
    cout << "Rectangle Name: RECTANGLE A\n" << "X-coordinate: 5\n" << "Y-coordinate: 7\n"
        << "Side a: 12\n" << "Side b: 15\n" << "Area: 180\n" << "Perimeter: 54\n" ;
    cout << "\nIf it doesn't there is a problem with your assignment operator.\n" << endl;
    rec2.display();

    cout << "\nTesting copy constructor in class Rectangle:" <<endl;

```

```

Rectangle rec3(a);
rec3.display();

a.set_side_b(300);
a.set_side_a(400);

cout << "\nExpected to display the following values for objec rec2: " << endl;
cout << "Rectangle Name: RECTANGLE A\n" << "X-coordinate: 5\n" << "Y-coordinate: 7\n"
<< "Side a: 100\n" << "Side b: 200\n" << "Area: 20000\n" << "Perimeter: 600\n" ;
cout << "\nIf it doesn't there is a problem with your assignment operator.\n" << endl;
rec3.display();
#endif // end of block to test Rectangle

#if 1 // Change 0 to 1 to test using array of pointer and polymorphism
cout << "\nTesting array of pointers and polymorphism:" << endl;
Shape *sh[4];
sh[0] = &s;
sh[1] = &rec1;
sh[2] = &rec3;
sh[3] = &b;

sh[0]->display();
sh[1]->display();
sh[2]->display();
sh[3]->display();
#endif // end of block to test array of pointer and polymorphism

#if 1
cout << "\nTesting Functions in class Circle:" << "\n";
Circle c (3, 5, 9, "CIRCLE C");
c.display();
cout << "the area of " << c.getName() << " is: " << c.area() << endl;
cout << "the perimeter of " << c.getName() << " is: " << c.perimeter() << endl;
d = a.distance(c);
cout << "\nThe distance between rectangle a and circle c is: " << d << "\n";
#endif

#if 1
CurveCut rc (6, 5, 10, 12, 9, "CurveCut rc");
rc.display();
cout << "the area of " << rc.getName() << " is: " << rc.area() << endl;
cout << "the perimeter of " << rc.getName() << " is: " << rc.perimeter() << endl;
d = rc.distance(c);
cout << "\nThe distance between rc and c is: " << d << "\n";
#endif

#if 1
// Using array of Shape pointers:

```

```

Shape* sh2[4];
sh2[0] = &s;
sh2[1] = &a;
sh2[2] = &c;
sh2[3] = &rc;
for (int i = 0; i < 4; i++) {
    sh2[i]->display();
    cout << "The area of " << sh2[i]->getName() << " is: " << sh2[i]->area() << endl;
    cout << "The perimeter of " << sh2[i]->getName()
        << " is: " << sh2[i]->perimeter() << endl << endl;
}

```

```

cout << "\nTesting copy constructor in class CurveCut:" << endl;
CurveCut cc = rc;
cc.display();

```

```

CurveCut cc2 (2, 5, 100, 12, 9, "CurveCut cc2");

```

```

cout << "\nTesting assignment operator in class CurveCut:" << endl;
cc2.display();
cc2 = cc;
cc2.display();
cc.display();
#endif
}

```

```

/*
 * File Name: circle.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 * - Sarvesh Vettrivelan
 * - Aayush Karthikeyan
 * Submission Date: Sept. 28, 2026
 */

```

```

#include "circle.h"
#include <iostream>
#define PI 3.14
using namespace std;

```

```

Circle::Circle(int x, int y, int r, const char*name)
    : Shape(x, y, name), radius(r){}

```

```

void Circle::setRadius(int r) {radius = r;}

```

```

int Circle::getRadius() const {return radius;}

```

```

int Circle::area() const {return (int)(PI * radius * radius);}

```

```
int Circle::perimeter() const {return (int)(2 * PI * radius);}
```

```
void Circle::display() const {  
    cout<<"Circle Name: "<<getName()<<endl;  
    getOrigin().display();  
    cout<<"Radius: "<<getRadius()<<"\n";  
    cout<<"Area: "<< area()<<"\n";  
    cout<<"Perimeter: "<< perimeter()<<"\n";  
}
```

```
/*
```

```
* File Name: circle.h
```

```
* Assignment: Lab 3 Exercise A
```

```
* Completed By:
```

```
* - Sarvesh Vettrivelan
```

```
* - Aayush Karthikeyan
```

```
* Submission Date: Sept. 28, 2026
```

```
*/
```

```
#ifndef CIRCLE_H
```

```
#define CIRCLE_H
```

```
#include "shape.h"
```

```
class Circle : virtual public Shape {  
private:
```

```
    int radius;
```

```
public:
```

```
    Circle(int x, int y, int r, const char*name);
```

```
    void setRadius(int r);
```

```
    int getRadius() const;
```

```
    virtual int area() const;
```

```
    virtual int perimeter() const;
```

```
    virtual void display() const;
```

```
};
```

```
#endif
```

```
/*
```

```
* File Name: curveCut.cpp
```

```
* Assignment: Lab 3 Exercise A
```

```
* Completed By:
```

```
* - Sarvesh Vettrivelan
```

```
* - Aayush Karthikeyan
```

```
* Submission Date: Sept. 28, 2026
```

```
*/
```

```

#include "curveCut.h"
#include <cmath>
#include <iostream>
#include <cstdlib>
using namespace std;

CurveCut::CurveCut(int x, int y, int width, int length, int curve_radius, const char* name)
    : Shape(x, y, name), Rectangle(x, y, width, length, name), Circle(x, y, curve_radius, name){
    check_dimensions(width, length, curve_radius);
}

CurveCut::CurveCut(const CurveCut& other)
    : Shape(other), Rectangle(other), Circle(other) {}

CurveCut& CurveCut::operator=(const CurveCut& rhs) {
    if(this != &rhs){
        Shape::operator=(rhs);
        Rectangle::set_side_a(rhs.get_side_a());
        Rectangle::set_side_b(rhs.get_side_b());
        Circle::setRadius(rhs.getRadius());
    }
    return *this;
}

void CurveCut::check_dimensions(int width, int length, int curve_radius) const {
    if (curve_radius > width || curve_radius > length){
        cout<<"\nCurve Radius cannot be greater than width or length"<<"\n";
        exit(EXIT_FAILURE);
    }
}

void CurveCut::setWidth(int width) {
    check_dimensions(width, get_side_b(), getRadius());
    Rectangle::set_side_a(width);
}

void CurveCut::setLength(int length){
    check_dimensions(get_side_a(), length, getRadius());
    Rectangle::set_side_b(length);
}

void CurveCut::setCurveRadius(int curve_radius){
    check_dimensions(get_side_a(), get_side_b(), curve_radius);
    Circle::setRadius(curve_radius);
}

int CurveCut::area() const {return ((Rectangle::area()) - (Circle::area()/4));}

```

```
int CurveCut::perimeter() const {return Rectangle::perimeter() - 2*getRadius() + Circle::perimeter()/4;}
```

```
void CurveCut::display() const {  
    cout << "CurveCut Name: " << getName() << '\n';  
    getOrigin().display();  
    cout << "Width: " << get_side_a() << '\n';  
    cout << "Length: " << get_side_b() << '\n';  
    cout << "Radius of the cut: " << getRadius() << '\n';
```

```
}
```

```
/*
```

```
* File Name: curveCut.h
```

```
* Assignment: Lab 3 Exercise A
```

```
* Completed By:
```

```
* - Sarvesh Vettrivelan
```

```
* - Aayush Karthikeyan
```

```
* Submission Date: Sept. 28, 2026
```

```
*/
```

```
#ifndef CURVECUT_H
```

```
#define CURVECUT_H
```

```
#include "rectangle.h"
```

```
#include "circle.h"
```

```
class CurveCut : public Rectangle, public Circle {
```

```
private:
```

```
    void check_dimensions(int width, int length, int curve_radius) const;
```

```
public:
```

```
    CurveCut(int x, int y, int width, int length, int curve_radius, const char* name);
```

```
    CurveCut(const CurveCut& other);
```

```
    CurveCut& operator=(const CurveCut& rhs);
```

```
    virtual void setWidth(int width);
```

```
    virtual void setLength(int length);
```

```
    virtual void setCurveRadius(int curve_radius);
```

```
    virtual int area() const;
```

```
    virtual int perimeter() const;
```

```
    virtual void display() const;
```

```
};
```

```
#endif
```

```
/*
```

```
* File Name: point.cpp
```

```
* Assignment: Lab 3 Exercise A
```

```
* Completed By:  
* - Sarvesh Vettrivelan  
* - Aayush Karthikeyan  
* Submission Date: Sept. 28, 2026  
*/
```

```
#include "point.h"  
#include <iostream>  
#include <cmath>  
#include <iomanip>  
using namespace std;
```

```
int Point::count = 0;  
int Point::nextId = 1001;
```

```
Point::Point(int x, int y): ptX(x), ptY(y), id(nextId){  
    count++;  
}
```

```
Point::Point(const Point& psrc): ptX(psrc.ptX), ptY(psrc.ptY), id(nextId++) {  
    count++;  
}
```

```
Point::~~Point(){  
    count--;  
}
```

```
Point& Point::operator=(const Point& rhs){  
    if (this != &rhs){  
        ptX = rhs.ptX;  
        ptY = rhs.ptY;  
    }  
    return *this;  
}
```

```
void Point::setX(int x){  
    ptX = x;  
}
```

```
void Point::setY(int y){  
    ptY = y;  
}
```

```
int Point::getX() const{  
    return ptX;  
}
```

```
int Point::getY() const{
```

```

        return ptY;
    }

int Point::counter(){
    return count;
}

double Point::distance(const Point& p1, const Point& p2){
    double xcord = p1.getX() - p2.getX();
    double ycord = p1.getY() - p2.getY();
    return sqrt((xcord * xcord) + (ycord * ycord));
}

double Point::distance(const Point& otherpt) const{
    return distance(*this, otherpt);
}

void Point::display() const{
    cout << "X-coordinate: " << fixed << setprecision(2) << ptX << endl;
    cout << "Y-coordinate: " << fixed << setprecision(2) << ptY << endl;
}
/*
* File Name: point.h
* Assignment: Lab 3 Exercise A
* Completed By:
*   - Sarvesh Vettrivelan
*   - Aayush Karthikeyan
* Submission Date: Sept. 28, 2026
*/

#ifndef POINT_H
#define POINT_H

class Point {
private:
    int ptX;
    int ptY;
    int id;
    static int count;
    static int nextId;

public:
    Point(int x, int y);
    Point(const Point& ptsrc);
    ~Point();
    Point& operator=(const Point& rhs);

    void display() const;

```

```

void setX(int x);
void setY(int y);
int getX() const;
int getY() const;

static int counter();

double distance(const Point& otherpt) const;
static double distance(const Point& p1, const Point& p2);
};

#endif
/*
* File Name: rectangle.cpp
* Assignment: Lab 3 Exercise A
* Completed By:
* - Sarvesh Vettrivelan
* - Aayush Karthikeyan
* Submission Date: Sept. 28, 2026
*/

#include "rectangle.h"
#include <iostream>
using namespace std;

Rectangle::Rectangle(int x, int y, int sideA, int sideB, const char* name)
    : Shape(x, y, name), Square(x, y, sideA, name), side_b(sideB){}

int Rectangle::get_side_b() const{return side_b;}

void Rectangle::set_side_b(int sideB){side_b = sideB;}

int Rectangle::area() const{return get_side_a() * side_b;}

int Rectangle::perimeter() const{return 2 * (get_side_a() + side_b);}

void Rectangle::display() const{
    cout<< "\nRectangle Name: " << getName() << "\n";
    getOrigin().display();
    cout << "Side a: " << get_side_a() << "\n";
    cout << "Side b: " << side_b << "\n";
    cout << "Area: " << area() << "\n";
    cout << "Perimeter: " << perimeter() << endl;
}
/*
* File Name: rectangle.h
* Assignment: Lab 3 Exercise A
* Completed By:

```

```
* - Sarvesh Vettrivelan
* - Aayush Karthikeyan
* Submission Date: Sept. 28, 2026
*/
```

```
#ifndef RECTANGLE_H
#define RECTANGLE_H
```

```
#include "square.h"
```

```
class Rectangle : public Square {
private:
    int side_b;
public:
    Rectangle(int x, int y, int sideA, int sideB, const char* name);
    int get_side_b() const;
    void set_side_b(int sideB);

    virtual int area() const override;
    virtual int perimeter() const override;
    virtual void display() const override;
};
```

```
#endif
```

```
/*
```

```
* File Name: shape.cpp
* Assignment: Lab 3 Exercise A
* Completed By:
* - Sarvesh Vettrivelan
* - Aayush Karthikeyan
* Submission Date: Sept. 28, 2026
*/
```

```
#include "shape.h"
#include <cstring>
#include <iostream>
using namespace std;
```

```
static char* copyString(const char* s){
    if (s == nullptr) s = "";
    char* p = new char[strlen(s) + 1];
    strcpy(p, s);
    return p;
}
```

```
Shape::Shape(int x, int y, const char* name): origin(x,y){
    shapeName = copyString(name);
}
```

```

Shape::Shape(const Shape& src): origin(src.origin){
    shapeName = copyString(src.shapeName);
}

Shape::~Shape(){
    delete[] shapeName;
}

Shape& Shape::operator=(const Shape& rhs){
    if(this != &rhs){
        char* newName = copyString(rhs.shapeName);
        delete[] shapeName;
        shapeName = newName;
        origin = rhs.origin;
    }
    return *this;
}

void Shape::display() const{
    cout<<"Shape Name: "<<shapeName<<endl;
    origin.display();
}

const Point& Shape::getOrigin() const{return origin;}

const char* Shape::getName() const {return shapeName;}

double Shape::distance(Shape& the_shape, Shape& other){
    return Point::distance(the_shape.origin, other.origin);
}

double Shape::distance(Shape& other){
    return Point::distance(origin, other.origin);
}

void Shape::move(double dx, double dy){
    origin.setX(static_cast<int>(origin.getX() + dx));
    origin.setY(static_cast<int>(origin.getY() + dy));
}
/*
 * File Name: shape.h
 * Assignment: Lab 3 Exercise A
 * Completed By:
 * - Sarvesh Vettrivelan
 * - Aayush Karthikeyan
 * Submission Date: Sept. 28, 2026
 */

```

```

#ifndef SHAPE_H
#define SHAPE_H

#include "point.h"

class Shape{
private:
    Point origin;
    char* shapeName;

public:
    Shape(int x, int y, const char* name);
    Shape(const Shape& src);
    virtual ~Shape();
    Shape& operator=(const Shape& rhs);
    virtual void display() const;

    const Point& getOrigin() const;
    const char* getName() const;

    virtual int area() const = 0;
    virtual int perimeter() const = 0;

    double distance(Shape& other);
    static double distance(Shape& the_shape, Shape& other);
    void move(double dx, double dy);

};
#endif
/*
 * File Name: square.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 28, 2026
 */

#include "square.h"
#include <iostream>
using namespace std;

Square::Square(int x, int y, int sideA, const char* name):Shape(x,y,name), side_a(sideA){}

int Square::get_side_a() const{return side_a;}

void Square::set_side_a(int sideA){side_a = sideA;}

```

```

int Square::area() const{return side_a * side_a;}

int Square::perimeter() const{return 4 * side_a;}

void Square::display() const{
    cout<< "Square Name: " << getName() << "\n";
    getOrigin().display();
    cout << "Side a: " << side_a << "\n";
    cout << "Area: " << area() << "\n";
    cout << "Perimeter: " << perimeter() << endl;
}
/*
* File Name: square.h
* Assignment: Lab 3 Exercise A
* Completed By:
*   - Sarvesh Vettrivelan
*   - Aayush Karthikeyan
* Submission Date: Sept. 28, 2026
*/

#ifndef SQUARE_H
#define SQUARE_H

#include "shape.h"

class Square : virtual public Shape {
private:
    int side_a;
public:
    Square(int x, int y, int sideA, const char* name);
    int get_side_a() const;
    void set_side_a(int sideA);

    virtual int area() const override;
    virtual int perimeter() const override;
    virtual void display() const;
};
#endif

```

OUTPUT:

```

user@DESKTOP-VTUJSUT /cygdrive/c/Users/user/OneDrive - University of Calgary/ENSF-480/ENSF480-
Labs/Lab-3/Ex-A
$ ./a.exe

```

Expected to display the distance between m and n is: 3

The distance between m and n is: 3

Expected second version of the distance function also print: 3

The distance between m and n is again: 3

Testing Functions in class Square:

Square Name: SQUARE - S

X-coordinate: 5

Y-coordinate: 7

Side a: 12

Area: 144

Perimeter: 48

Testing Functions in class Rectangle:

Rectangle Name: RECTANGLE A

X-coordinate: 5

Y-coordinate: 7

Side a: 12

Side b: 15

Area: 180

Perimeter: 54

Rectangle Name: RECTANGLE B

X-coordinate: 16

Y-coordinate: 7

Side a: 8

Side b: 9

Area: 72

Perimeter: 34

Distance between square a, and b is: 11.00

Rectangle Name: RECTANGLE A

X-coordinate: 5

Y-coordinate: 7

Side a: 12

Side b: 15

Area: 180

Perimeter: 54

Testing assignment operator in class Rectangle:

Rectangle Name: RECTANGLE rec2

X-coordinate: 3

Y-coordinate: 4

Side a: 11

Side b: 7

Area: 77

Perimeter: 36

Expected to display the following values for objec rec2:

Rectangle Name: RECTANGLE A

X-coordinate: 5

Y-coordinate: 7
Side a: 12
Side b: 15
Area: 180
Perimeter: 54

If it doesn't there is a problem with your assignment operator.

Rectangle Name: RECTANGLE A
X-coordinate: 5
Y-coordinate: 7
Side a: 12
Side b: 15
Area: 180
Perimeter: 54

Testing copy constructor in class Rectangle:

Rectangle Name: RECTANGLE A
X-coordinate: 5
Y-coordinate: 7
Side a: 100
Side b: 200
Area: 20000
Perimeter: 600

Expected to display the following values for objec rec2:

Rectangle Name: RECTANGLE A
X-coordinate: 5
Y-coordinate: 7
Side a: 100
Side b: 200
Area: 20000
Perimeter: 600

If it doesn't there is a problem with your assignment operator.

Rectangle Name: RECTANGLE A
X-coordinate: 5
Y-coordinate: 7
Side a: 100
Side b: 200
Area: 20000
Perimeter: 600

Testing array of pointers and polymorphism:

Square Name: SQUARE - S

X-coordinate: 5
Y-coordinate: 7
Side a: 12
Area: 144
Perimeter: 48

Rectangle Name: RECTANGLE A
X-coordinate: 5
Y-coordinate: 7
Side a: 12
Side b: 15
Area: 180
Perimeter: 54

Rectangle Name: RECTANGLE A
X-coordinate: 5
Y-coordinate: 7
Side a: 100
Side b: 200
Area: 20000
Perimeter: 600

Rectangle Name: RECTANGLE B
X-coordinate: 16
Y-coordinate: 7
Side a: 8
Side b: 9
Area: 72
Perimeter: 34

Testing Functions in class Circle:
Circle Name: CIRCLE C
X-coordinate: 3
Y-coordinate: 5
Radius: 9
Area: 254
Perimeter: 56
the area of CIRCLE C is: 254
the perimeter of CIRCLE C is: 56

The distance between rectangle a and circle c is: 2.83
CurveCut Name: CurveCut rc
X-coordinate: 6
Y-coordinate: 5
Width: 10
Length: 12
Radius of the cut: 9
the area of CurveCut rc is: 57

the perimeter of CurveCut rc is: 40

The distance between rc and c is: 3.00

Square Name: SQUARE - S

X-coordinate: 5

Y-coordinate: 7

Side a: 12

Area: 144

Perimeter: 48

The area of SQUARE - S is: 144

The perimeter of SQUARE - S is: 48

Rectangle Name: RECTANGLE A

X-coordinate: 5

Y-coordinate: 7

Side a: 400

Side b: 300

Area: 120000

Perimeter: 1400

The area of RECTANGLE A is: 120000

The perimeter of RECTANGLE A is: 1400

Circle Name: CIRCLE C

X-coordinate: 3

Y-coordinate: 5

Radius: 9

Area: 254

Perimeter: 56

The area of CIRCLE C is: 254

The perimeter of CIRCLE C is: 56

CurveCut Name: CurveCut rc

X-coordinate: 6

Y-coordinate: 5

Width: 10

Length: 12

Radius of the cut: 9

The area of CurveCut rc is: 57

The perimeter of CurveCut rc is: 40

Testing copy constructor in class CurveCut:

CurveCut Name: CurveCut rc

X-coordinate: 6

Y-coordinate: 5

Width: 10

Length: 12

Radius of the cut: 9

Testing assignment operator in class CurveCut:

CurveCut Name: CurveCut cc2

X-coordinate: 2

Y-coordinate: 5

Width: 100

Length: 12

Radius of the cut: 9

CurveCut Name: CurveCut rc

X-coordinate: 6

Y-coordinate: 5

Width: 10

Length: 12

Radius of the cut: 9

CurveCut Name: CurveCut rc

X-coordinate: 6

Y-coordinate: 5

Width: 10

Length: 12

Radius of the cut: 9

Exercise-B:

```
/*  
 * File Name: iterator.cpp  
 * Assignment: Lab 3 Exercise B  
 * Completed By:  
 *   - Sarvesh Vettrivelan  
 *   - Aayush Karthikeyan  
 * Submission Date: Sept. 28, 2026  
 */
```

```
#include <iostream>  
#include <assert.h>  
#include "mystring2.h"  
#include <cstring>
```

```
using namespace std;
```

```
template <class T>  
class Vector {  
public:  
  
    class VectIter {  
        friend class Vector<T>;  
    private:  
        Vector<T>* v; // points to a vector object of type T  
        int index;    // represents the subscript number of the vector's  
                      // array.
```

```

public:
    VectIter(Vector<T>& x);

    T operator++();
    // PROMISES: increments the iterator's index and return the
    //     value of the element at the index position. If
    //     index exceeds the size of the array it will
    //     be set to zero. Which means it will be circulated
    //     back to the first element of the vector.

    T operator++(int);
    // PROMISES: returns the value of the element at the index
    //     position, then increments the index. If
    //     index exceeds the size of the array it will
    //     be set to zero. Which means it will be circulated
    //     back to the first element of the vector.

    T operator--();
    // PROMISES: decrements the iterator index, and return the
    //     the value of the element at the index. If
    //     index is less than zero it will be set to the
    //     last element in the array. Which means it will be
    //     circulated to the last element of the vector.

    T operator--(int);
    // PROMISES: returns the value of the element at the index
    //     position, then decrements the index. If
    //     index is less than zero it will be set to the
    //     last element in the array. Which means it will be
    //     circulated to the last element of the vector.

    T operator *() const;
    // PROMISES: returns the value of the element at the current
    //     index position.
};

Vector(int sz);
~Vector();

T& operator[](int i);
// PROMISES: returns existing value in the ith element of
//     array or sets a new value to the ith element in
//     array.

void ascending_sort();
// PROMISES: sorts the vector values in ascending order.

private:

```

```

T *array;          // points to the first element of an array of T
int size;          // size of array
void swap(T& , T& ); // swaps the values of two elements in array
public:
};

```

```

template <class T>
void Vector<T>::ascending_sort()
{
    for(int i=0; i< size-1; i++)
        for(int j=i+1; j < size; j++)
            if(array[i] > array[j])
                swap(array[i], array[j]);
}

```

```

template <>
void Vector<const char*>::ascending_sort()
{
    for(int i=0; i< size-1; i++)
        for(int j=i+1; j < size; j++)
            if(strcmp(array[i],array[j]) > 0)
                swap(array[i], array[j]);
}

```

```

template <>
void Vector<Mystring>::ascending_sort()
{
    for(int i=0; i< size-1; i++)
        for(int j=i+1; j < size; j++)
            if(strcmp(array[i].c_str(), array[j].c_str()) > 0)
                swap(array[i], array[j]);
}

```

```

template <class T>
void Vector<T>::swap(T& a, T& b)
{
    T tmp = a;
    a = b;
    b = tmp;
}

```

```

template <class T>
T Vector<T>::VectIter::operator *() const
{
    return v -> array[index];
}

```

```

template <class T>

```

```

Vector<T>::VectIter::VectIter(Vector<T>& x)
{
    v = &x;
    index = 0;
}

```

```

template <class T>
Vector<T>::Vector(int sz)
{
    size=sz;
    array = new T [sz];
    assert (array != NULL);
}

```

```

template <class T>
Vector<T>::~~Vector()
{
    delete [] array;
    array = NULL;
}

```

```

template <class T>
T& Vector<T>::operator [] (int i)
{
    return array[i];
}

```

```

template <class T>
T Vector<T>::VectIter::operator++() {
    index++;
    if(index >= v->size){
        index = 0;
    }
    return v->array[index];
}

```

```

// template <class T>
// T Vector<T>::VectIter::operator++(int) {
//     int i = index;
//     index++;
//     if(index >= v->size){
//         index = 0;
//     }
//     return v->array[i];
// }

```

```

// another method

```

```

template <class T>
T Vector<T>::VectIter::operator++(int) {
    T value = v->array[index];
    ++(*this);
    return value;
}

```

```

template <class T>
T Vector<T>::VectIter::operator--() {
    index--;
    if(index < 0){
        index = v->size - 1 ;
    }
    return v->array[index];
}

```

```

template <class T>
T Vector<T>::VectIter::operator--(int) {
    T value = v->array[index];
    --(*this);
    return value;
}

```

```

int main()
{

```

```

    Vector <int> x(3);
    x[0] = 999;
    x[1] = -77;
    x[2] = 88;

```

```

    Vector <int>::VectIter iter(x);

```

```

    cout << "\n\nThe first element of vector x contains: " << *iter;

```

```

    // the code between the #if 0 and #endif is ignored by
    // compiler. If you change it to #if 1, it will be compiled

```

```

#if 1
    cout << "\nTesting an <int> Vector: " << endl;

```

```

    cout << "\n\nTesting sort";
    x.sort();

```

```

    for (int i=0; i<3 ; i++)
        cout << endl << iter++;

```

```

    cout << "\n\nTesting Prefix --:";

```

```

for (int i=0; i<3 ; i++)
    cout << endl << --iter;

cout << "\n\nTesting Prefix ++:";
for (int i=0; i<3 ; i++)
    cout << endl << ++iter;

cout << "\n\nTesting Postfix --";
for (int i=0; i<3 ; i++)
    cout << endl << iter--;

cout << endl;

cout << "Testing a <Mystring> Vector: " << endl;
Vector<Mystring> y(3);
y[0] = "Bar";
y[1] = "Foo";
y[2] = "All";;

Vector<Mystring>::VectIter iters(y);

cout << "\n\nTesting sort";
y ascending_sort();

for (int i=0; i<3 ; i++)
    cout << endl << iters++;

cout << "\n\nTesting Prefix --:";
for (int i=0; i<3 ; i++)
    cout << endl << --iters;

cout << "\n\nTesting Prefix ++:";
for (int i=0; i<3 ; i++)
    cout << endl << ++iters;

cout << "\n\nTesting Postfix --";
for (int i=0; i<3 ; i++)
    cout << endl << iters--;

cout << endl; cout << "Testing a <char *> Vector: " << endl;
Vector<const char*> z(3);
z[0] = "Orange";
z[1] = "Pear";
z[2] = "Apple";;

Vector<const char*>::VectIter iterchar(z);

cout << "\n\nTesting sort";

```

```

z.ascending_sort();

for (int i=0; i<3 ; i++)
    cout << endl << iterchar++;

#endif
cout << "\nPrgram Terminated Successfully." << endl;

return 0;
}

/*
 * File Name: mystring2.cpp
 * Assignment: Lab 3 Exercise B
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 28, 2026
 */

#include "mystring2.h"
#include <string.h>
#include <iostream>
using namespace std;

Mystring::Mystring()
{
    charsM = new char[1];
    charsM[0] = '\0';
    lengthM = 0;
}

Mystring::Mystring(const char *s)
    : lengthM((int)strlen(s))
{
    charsM = new char[lengthM + 1];
    strcpy(charsM, s);
}

Mystring::Mystring(int n)
    : lengthM(0), charsM(new char[n])
{
    charsM[0] = '\0';
}

Mystring::Mystring(const Mystring& source):
    lengthM(source.lengthM), charsM(new char[source.lengthM+1])
{

```

```

    strcpy (charsM, source.charsM);
}

Mystring::~Mystring()
{
    delete [] charsM;
}

int Mystring::length() const
{
    return lengthM;
}

char Mystring::get_char(int pos) const
{
    if(pos < 0 && pos >= length()){
        cerr << "\nERROR: get_char: the position is out of boundary." ;
    }

    return charsM[pos];
}

const char * Mystring::c_str() const
{
    return charsM;
}

void Mystring::set_char(int pos, char c)
{
    if(pos < 0 && pos >= length()){
        cerr << "\nset_char: the position is out of boundary."
        << " Nothing was changed.";
        return;
    }

    if (c != '\0'){
        cerr << "\nset_char: char c is empty."
        << " Nothing was changed.";
        return;
    }

    charsM[pos] = c;
}

Mystring& Mystring::operator =(const Mystring& S)
{
    if(this == &S)
        return *this;

```

```

delete [] charsM;
lengthM = (int) strlen(S.charsM);
charsM = new char [lengthM+1];
strcpy(charsM,S.charsM);
return *this;
}

```

```

Mystring& Mystring::append(const Mystring& other)
{
    char *tmp = new char [lengthM + other.lengthM + 1];
    lengthM+=other.lengthM;
    strcpy(tmp, charsM);
    strcat(tmp, other.charsM);
    delete []charsM;
    charsM = tmp;

    return *this;
}

```

```

void Mystring::set_str(char* s)
{
    delete []charsM;
    lengthM = (int) strlen(s);
    charsM=new char[lengthM+1];

    strcpy(charsM, s);
}

```

```

std::ostream& operator <<(std::ostream& os, const Mystring& s)
{
    return os << s.charsM;
}
/*

```

```

* File Name: mystring2.h
* Assignment: Lab 3 Exercise B
* Completed By:
*   - Sarvesh Vettrivelan
*   - Aayush Karthikeyan
* Submission Date: Sept. 28, 2026
*/

```

```

#ifndef MYSTRING_H
#define MYSTRING_H
#include <iostream>
class Mystring {
    friend std::ostream& operator <<(std::ostream& os, const Mystring& s);
public:
    Mystring();

```

```

// PROMISES: Empty string object is created.

Mystring(int n);
// PROMISES: Creates an empty string with a total capacity of n.
//      In other words, dynamically allocates n elements for
//      charsM, sets the lengthM to zero, and fills the first
//      element of charsM with '\0'.

Mystring(const char *s);
// REQUIRES: s points to first char of a built-in string.
// REQUIRES: Mystring object is created by copying chars from s.

~Mystring(); // destructor

Mystring(const Mystring& source); // copy constructor

Mystring& operator=(const Mystring& rhs); // assignment operator
// REQUIRES: rhs is reference to a Mystring as a source
// PROMISES: to make this-object (object that this is pointing to, as a copy
//      of rhs.

int length() const;
// PROMISES: Return value is number of chars in charsM.

char get_char(int pos) const;
// REQUIRES: pos >= 0 && pos < length()
// PROMISES:
//      Return value is char at position pos.
//      (The first char in the charsM is at position 0.)

const char * c_str() const;
// PROMISES:
//      Return value points to first char in built-in string
//      containing the chars of the string object.

void set_char(int pos, char c);
// REQUIRES: pos >= 0 && pos < length(), c != '\0'
// PROMISES: Character at position pos is set equal to c.

Mystring& append(const Mystring& other);

// PROMISES: extends the size of charsM to allow concatenate other.charsM to
//      to the end of charsM. For example if charsM points to "ABC", and
//      other.charsM points to XYZ, extends charsM to "ABCXYZ".
//

void set_str(char* s);
// REQUIRES: s is a valid C++ string of characters (a built-in string)

```

```
// PROMISES: copies s into charsM, if the length of s is less than or equal lengthM.
//      Othrewise, extends the size of the charsM to s.lengthM+1, and copies
//      s into the charsM.
```

private:

```
int lengthM; // the string length - number of characters excluding \0
char* charsM; // a pointer to the beginning of an array of characters, allocated dynamically.
void memory_check(char* s);
// PROMISES: if s points to NULL terminates the program.
};
```

#endif

OUTPUT:

```
user@DESKTOP-VTUJSUT /cygdrive/c/Users/user/OneDrive - University of Calgary/ENSF-480/ENSF480-Labs/Lab-3/Ex-B
$ ./a.exe

The first element of vector x contains: 999
Testing an <int> Vector:

Testing sort
-77
88
999

Testing Prefix --:
999
88
-77

Testing Prefix ++:
88
999
-77

Testing Postfix --
-77
999
88
Testing a <Mystring> Vector:

Testing sort
All
Bar
Foo

Testing Prefix --:
Foo
Bar
All

Testing Prefix ++:
Bar
Foo
All

Testing Postfix --
All
Foo
Bar
Testing a <char *> Vector:

Testing sort
Apple
Orange
Pear
Program Terminated Successfully.
```

Exercise-C:

```
/*
 * File Name: customer.cpp
 * Assignment: Lab 3 Exercise C
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 28, 2026
 */

#include "customer.h"
#include <iostream>

using namespace std;

Customer::Customer(const Mystring& fn, const Mystring& ln, const Mystring& ad, const Mystring&
ph):fname(fn), lname(ln), address(ad), phone(ph)
{
}

const char* Customer::getFname()const
{
    return fname.c_str();
}

const char* Customer::getLname()const
{
    return lname.c_str();
}

const char* Customer::getAddress()const
{
    return address.c_str();
}

const char* Customer::getPhone()const
{
    return phone.c_str();
}

void Customer::setFname(const char*s)
{
    Mystring tmp(s);
    fname=tmp;
}

void Customer::setLname(const char*s)
{

```

```

    Mystring tmp(s);
    lname=tmp;
}

void Customer::setAddress(const char*s)
{
    Mystring tmp(s);
    address=tmp;
}

void Customer::setPhone(const char*s)
{
    Mystring tmp(s);
    phone=tmp;
}

ostream& operator << (ostream& os, const Customer& c)
{
    os<< "Name: " << c.getFname();
    os<< " " << c.getLname();
    os<< ". Address: " << c.getAddress();
    os<< ". Phone: " << c.getPhone();
    return os;
}
/*
* File Name: customer.h
* Assignment: Lab 3 Exercise C
* Completed By:
*   - Sarvesh Vettrivelan
*   - Aayush Karthikeyan
* Submission Date: Sept. 28, 2026
*/

#ifndef CUSTOMER
#define CUSTOMER
#include <iostream>

#include "mystring2.h"

using namespace std;

class Customer{
friend ostream& operator << (ostream&, const Customer& c);

public:
    Customer(const Mystring& fn, const Mystring& ln, const Mystring& ad, const Mystring& ph);
    const char* getFname() const;
    const char* getLname()const;

```

```
const char* getAddress()const;
const char* getPhone()const;
void setFname(const char*s);
void setLname(const char*s);
void setAddress(const char*s);
void setPhone(const char*s);
```

```
private:
    Mystring fname;
    Mystring lname;
    Mystring address;
    Mystring phone;
};
```

```
#endif
```

```
/*
 * File Name: lookupTable.h
 * Assignment: Lab 3 Exercise C
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 28, 2026
 */
```

```
#ifndef LOOKUPTABLE_H
#define LOOKUPTABLE_H
#include <iostream>
#include <cassert>
using namespace std;
```

```
// class LookupTable: GENERAL CONCEPTS
//
// key/datum pairs are ordered. The first pair is the pair with
// the lowest key, the second pair is the pair with the second
// lowest key, and so on. This implies that you must be able to
// compare two keys with the < operator.
//
// Each LookupTable has an embedded iterator class that allows users
// of the class to traverse through the list and have access to each
// node.
```

```
#include "customer.h"
```

```
// In this version of the LookupTable a new struct type called Pair
// is introduced which represents a key/data pair.
```

```
template<class LT_Key, class LT_Datum>
```

```

struct Pair
{
    Pair(LT_Key keyA, LT_Datum datumA):key(keyA), datum(datumA)
    {
    }

    LT_Key key;
    LT_Datum datum;
};

```

```

template<class LT_Key, class LT_Datum>
class LookupTable;

```

```

template<class LT_Key, class LT_Datum>
class LT_Node {
    friend class LookupTable<LT_Key, LT_Datum>;
private:
    Pair<LT_Key, LT_Datum> pairM;
    LT_Node* nextM;

    // This ctor should be convenient in insert and copy operations.
    LT_Node(const Pair<LT_Key, LT_Datum>& pairA, LT_Node* nextA);
};

```

```

template<class LT_Key, class LT_Datum>
class LookupTable {
public:

    // Nested class
    class Iterator {
        friend class LookupTable ;
        LookupTable* LT;
    //   LT_Node* cursor;

```

```

public:
    Iterator():LT(0){}
    Iterator(LookupTable & x): LT(&x){}
    const LT_Datum& operator *();
    const LT_Datum& operator ++();
    const LT_Datum& operator ++(int);
    int operator !();

    void step_fwd(){
        assert(LT->cursor_ok());
        LT->step_fwd();
    }
};

```

```

LookupTable();
LookupTable(const LookupTable & source);
LookupTable& operator =(const LookupTable& rhs);
~LookupTable();

LookupTable& begin();

int size() const;
// PROMISES: Returns number of keys in the table.

int cursor_ok() const;
// PROMISES:
// Returns 1 if the cursor is attached to a key/datum pair,
// and 0 if the cursor is in the off-list state.

const LT_Key& cursor_key() const;
// REQUIRES: cursor_ok()
// PROMISES: Returns key of key/datum pair to which cursor is attached.

const LT_Datum& cursor_datum() const;
// REQUIRES: cursor_ok()
// PROMISES: Returns datum of key/datum pair to which cursor is attached.

void insert(const Pair<LT_Key, LT_Datum>& pariA);
// PROMISES:
// If keyA matches a key in the table, the datum for that
// key is set equal to datumA.
// If keyA does not match an existing key, keyA and datumM are
// used to create a new key/datum pair in the table.
// In either case, the cursor goes to the off-list state.

void remove(const LT_Key& keyA);
// PROMISES:
// If keyA matches a key in the table, the corresponding
// key/datum pair is removed from the table.
// If keyA does not match an existing key, the table is unchanged.
// In either case, the cursor goes to the off-list state.

void find(const LT_Key& keyA);
// PROMISES:
// If keyA matches a key in the table, the cursor is attached
// to the corresponding key/datum pair.
// If keyA does not match an existing key, the cursor is put in
// the off-list state.

void go_to_first();
// PROMISES: If size() > 0, cursor is moved to the first key/datum pair
// in the table.

```

```

void step_fwd();
// REQUIRES: cursor_ok()
// PROMISES:
// If cursor is at the last key/datum pair in the list, cursor
// goes to the off-list state.
// Otherwise the cursor moves forward from one pair to the next.

void make_empty();
// PROMISES: size() == 0.

template<class K, class D>
friend ostream& operator << (ostream& os, const LookupTable<K, D>& lt);

private:
int sizeM;
LT_Node<LT_Key, LT_Datum>* headM;
LT_Node<LT_Key, LT_Datum>* cursorM;

void destroy();
// Deallocate all nodes, set headM to zero.

void copy(const LookupTable& source);
// Establishes *this as a copy of source. Cursor of *this will
// point to the twin of whatever the source's cursor points to.
};

template<class LT_Key, class LT_Datum>
LookupTable<LT_Key, LT_Datum>& LookupTable<LT_Key, LT_Datum>::begin(){
    cursorM = headM;
    return *this;
}

template<class LT_Key, class LT_Datum>
LT_Node<LT_Key, LT_Datum>::LT_Node(const Pair<LT_Key, LT_Datum>& pairA, LT_Node *nextA)
: pairM(pairA), nextM(nextA)
{
}

template<class LT_Key, class LT_Datum>
LookupTable<LT_Key, LT_Datum>::LookupTable()
: sizeM(0), headM(0), cursorM(0)
{
}

template<class LT_Key, class LT_Datum>
LookupTable<LT_Key, LT_Datum>::LookupTable(const LookupTable& source)
{

```

```

    copy(source);
}

template<class LT_Key, class LT_Datum>
LookupTable<LT_Key, LT_Datum>& LookupTable<LT_Key, LT_Datum>::operator =(const LookupTable& rhs)
{
    if (this != &rhs) {
        destroy();
        copy(rhs);
    }
    return *this;
}

template<class LT_Key, class LT_Datum>
LookupTable<LT_Key, LT_Datum>::~~LookupTable()
{
    destroy();
}

template<class LT_Key, class LT_Datum>
int LookupTable<LT_Key, LT_Datum>::size() const
{
    return sizeM;
}

template<class LT_Key, class LT_Datum>
int LookupTable<LT_Key, LT_Datum>::cursor_ok() const
{
    return cursorM != 0;
}

template<class LT_Key, class LT_Datum>
const LT_Key& LookupTable<LT_Key, LT_Datum>::cursor_key() const
{
    assert(cursor_ok());
    return cursorM->pairM.key;
}

template<class LT_Key, class LT_Datum>
const LT_Datum& LookupTable<LT_Key, LT_Datum>::cursor_datum() const
{
    assert(cursor_ok());
    return cursorM->pairM.datum;
}

template<class LT_Key, class LT_Datum>
void LookupTable<LT_Key, LT_Datum>::insert(const Pair<LT_Key, LT_Datum>& pairA)
{

```

```

// Add new node at head?
if (headM == 0 || pairA.key < headM->pairM.key) {
    headM = new LT_Node<LT_Key, LT_Datum>(pairA, headM);
    sizeM++;
}

// Overwrite datum at head?
else if (pairA.key == headM->pairM.key)
    headM->pairM.datum = pairA.datum;

// Have to search ...

else {
    LT_Node<LT_Key, LT_Datum>* before= headM;
    LT_Node<LT_Key, LT_Datum>* after=headM->nextM;

    while(after!=NULL && (pairA.key > after->pairM.key))
    {
        before=after;
        after=after->nextM;
    }

    if(after!=NULL && pairA.key == after->pairM.key)
    {
        after->pairM.datum = pairA.datum;
    }
    else
    {
        before->nextM = new LT_Node<LT_Key, LT_Datum>(pairA, before->nextM);
        sizeM++;
    }
}

template<class LT_Key, class LT_Datum>
void LookupTable<LT_Key, LT_Datum>::remove(const LT_Key& keyA)
{
    if (headM == 0 || keyA < headM->pairM.key)
        return;

    LT_Node<LT_Key, LT_Datum>* doomed_node = 0;
    if (keyA == headM->pairM.key) {
        doomed_node = headM;
        headM = headM->nextM;
        sizeM--;
    }
    else {

```

```

    LT_Node<LT_Key, LT_Datum>* before = headM;
    LT_Node<LT_Key, LT_Datum>* maybe_doomed = headM->nextM;
    while(maybe_doomed != 0 && keyA > maybe_doomed->pairM.key) {
        before = maybe_doomed;
        maybe_doomed = maybe_doomed->nextM;
    }

    if (maybe_doomed != 0 && maybe_doomed->pairM.key == keyA) {
        doomed_node = maybe_doomed;
        before->nextM = maybe_doomed->nextM;
        sizeM--;
    }
}
delete doomed_node;    // Does nothing if doomed_node == 0.
}

```

```

template<class LT_Key, class LT_Datum>
void LookupTable<LT_Key, LT_Datum>::find(const LT_Key& keyA)
{
    LT_Node<LT_Key, LT_Datum> *ptr=headM;
    while (ptr != NULL && ptr->pairM.key != keyA)
    {
        ptr=ptr->nextM;
    }

    cursorM = ptr;
}

```

```

template<class LT_Key, class LT_Datum>
void LookupTable<LT_Key, LT_Datum>::go_to_first()
{
    cursorM = headM;
}

```

```

template<class LT_Key, class LT_Datum>
void LookupTable<LT_Key, LT_Datum>::step_fwd()
{
    assert(cursor_ok());
    cursorM = cursorM->nextM;
}

```

```

template<class LT_Key, class LT_Datum>
void LookupTable<LT_Key, LT_Datum>::make_empty()
{
    destroy();
    sizeM = 0;
    cursorM = 0;
}

```

```

template<class LT_Key, class LT_Datum>
void LookupTable<LT_Key, LT_Datum>::destroy()
{

    LT_Node<LT_Key, LT_Datum> *ptr = headM;
    while (ptr!=NULL)
    {
        headM=headM->nextM;
        delete ptr;
        ptr=headM;
    }
    cursorM = NULL;
    sizeM=0;
}

template<class LT_Key, class LT_Datum>
void LookupTable<LT_Key, LT_Datum>::copy(const LookupTable& source)
{
    sizeM = 0;
    headM=0;
    cursorM =0;

    if(source.headM ==0)
        return;

    for(LT_Node<LT_Key, LT_Datum> *p = source.headM; p != 0; p=p->nextM)
    {
        insert(Pair<LT_Key, LT_Datum>(p->pairM.key, p->pairM.datum));
        if(source.cursorM == p)
            find(p->pairM.key);
    }

}

template<class K, class D>
ostream& operator << (ostream& os, const LookupTable<K, D>& lt)
{
    if (lt.cursor_ok())
        os <<lt.cursor_key() << " " << lt.cursor_datum();
    else
        os<<"Not Found.";

    return os;
}

template<class LT_Key, class LT_Datum>

```

```

const LT_Datum& LookupTable<LT_Key, LT_Datum>::Iterator::operator *()
{
    assert(LT->cursor_ok());
    return LT->cursor_datum();
}

template<class LT_Key, class LT_Datum>
const LT_Datum& LookupTable<LT_Key, LT_Datum>::Iterator::operator ++()
{
    assert(LT->cursor_ok());
    const LT_Datum & x = LT->cursor_datum();
    LT->step_fwd();
    return x;
}

template<class LT_Key, class LT_Datum>
const LT_Datum& LookupTable<LT_Key, LT_Datum>::Iterator::operator ++(int)
{
    assert(LT->cursor_ok());

    LT->step_fwd();
    return LT->cursor_datum();
}

template<class LT_Key, class LT_Datum>
int LookupTable<LT_Key, LT_Datum>::Iterator::operator !()
{
    return (LT->cursor_ok());
}

#endif

/*
 * File Name: mainLab3ExC.cpp
 * Assignment: Lab 3 Exercise C
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 28, 2026
 */

#include <assert.h>
#include <iostream>
#include "lookupTable.h"
#include "customer.h"
#include <cstring>
using namespace std;

```

```

template<class LT_Key, class LT_Datum>
void print(LookupTable<LT_Key, LT_Datum>& lt);

template<class LT_Key, class LT_Datum>
void try_to_find(LookupTable<LT_Key, LT_Datum>& lt, LT_Key key);

void test_Customer();

//Uncomment the following function calls when ready to test template class LookupTable
void test_String();
void test_integer();

int main()
{

//create and test a lookup table with an integer key value and Customer datum
test_Customer();

// Uncomment the following function calls when ready to test template class LookupTable
// create and test a a lookup table of type <int, String>
test_String();

// Uncomment the following function calls when ready to test template class LookupTable
// create and test a a lookup table of type <int, int>
test_integer();

cout<<"\n\nProgram terminated successfully.\n\n";

return 0;
}

template<class LT_Key, class LT_Datum>
void print(LookupTable<LT_Key, LT_Datum>& lt)
{
if (lt.size() == 0)
    cout << " Table is EMPTY.\n";
for (lt.go_to_first(); lt.cursor_ok(); lt.step_fwd()) {
    cout << lt << endl;
}
}

template<class LT_Key, class LT_Datum>
void try_to_find(LookupTable<LT_Key, LT_Datum>& lt, LT_Key key)
{
    lt.find(key);
    if (lt.cursor_ok())
        cout << "\nFound key:" << lt;
    else

```

```

    cout << "\nSorry, I couldn't find key: " << key << " in the table.\n";
}

void test_Customer()
//creating a lookup table for customer objects.
{
    cout<<"\nCreating and testing Customers Lookup Table <template>-...\n";
    LookupTable<int, Customer> lt;

    // Insert using new keys.
    Customer a("Joe", "Morrison", "11 St. Calgary.", "(403)-1111-123333");
    Customer b("Jack", "Lewis", "12 St. Calgary.", "(403)-1111-123334");
    Customer c("Tim", "Hardy", "13 St. Calgary.", "(403)-1111-123335");
    lt.insert(Pair<int, Customer>(8002, a));
    lt.insert(Pair<int, Customer>(8004,c));
    lt.insert(Pair<int, Customer>(8001,b));

    assert(lt.size() == 3);
    lt.remove(8004);
    assert(lt.size() == 2);
    cout << "\nPrinting table after inserting 3 new keys and 1 removal...\n";
    print(lt);

    // Pretend that a user is trying to look up customers info.

    cout << "\nLet's look up some names ...\n";
    try_to_find(lt, 8001);
    try_to_find(lt, 8000);

    // test Iterator
    cout << "\nTesting and using iterator ...\n";
    LookupTable<int, Customer>::Iterator it = lt.begin();
    cout << "\nThe first node contains: " <<*it <<endl;

    while (!it) {
        cout << ++it << endl;
    }

    //test copying
    lt.go_to_first();
    lt.step_fwd();
    LookupTable<int, Customer> clt(lt);
    assert(strcmp(clt.cursor_datum().getFname(),"Joe")==0);

    cout << "\nTest copying: keys should be 8001, and 8002\n";
    print(clt);
    lt.remove(8002);

```

```

//Assignment operator check.
clt= lt;

cout << "\nTest assignment operator: key should be 8001\n";
print(clt);

//Wipe out the entries in the table.
lt.make_empty();
cout << "\nPrinting table for the last time: Table should be empty...\n";
print(lt);

cout << "****----Finished tests on Customers Lookup Table <template>-----****\n";
cout << "PRESS RETURN TO CONTINUE.";
cin.get();

}

/* Uncomment and modify the following function when ready to test LookupTable<int,Mystring>*/

void test_String()

// creating lookuptable for Mystring objects
{
    cout<<"\nCreating and testing LookupTable <int, Mystring> ..... \n";
    LookupTable<int, Mystring> lt;

    // Insert using new keys.

    Mystring a("I am an ENEL-409 student.");
    Mystring b("C++ is a powerful language for engineers but it's not easy.");
    Mystring c ("Winter 2004");

    lt.insert(Pair<int, Mystring> (8002,a));
    lt.insert(Pair<int, Mystring> (8001,b));
    lt.insert(Pair<int, Mystring> (8004,c));

    assert(lt.size() == 3);
    lt.remove(8004);
    assert(lt.size() == 2);
    cout << "\nPrinting table after inserting 3 new keys and 1 removal...\n";
    print(lt);

    // Pretend that a user is trying to look up customers info.

    cout << "\nLet's look up some names ... \n";
    try_to_find(lt, 8001);
    try_to_find(lt, 8000);

```

```

// test Iterator
LookupTable<int, Mystring> ::Iterator it = lt.begin();
cout <<"\nThe first node contains: " <<*it <<endl;

while (!it) {
    cout << ++it << endl;
}

//test copying
lt.go_to_first();
lt.step_fwd();
LookupTable<int, Mystring>  clt(lt);
assert(strcmp(clt.cursor_datum().c_str(),"I am an ENEL-409 student.")==0);

cout << "\nTest copying: keys should be 8001, and 8002\n";
print(clt);
lt.remove(8002);

//Assignment operator check.
clt= lt;

cout << "\nTest assignment operator: key should be 8001\n";
print(clt);

// Wipe out the entries in the table.
lt.make_empty();
cout << "\nPrinting table for the last time: Table should be empty ...\n";
print(lt);

cout << "***----Finished Lab 4 tests on <int> <Mystring>-----***\n";
cout << "PRESS RETURN TO CONTINUE.";
cin.get();
}

/* Uncomment and modify the following funciton when ready to test LookupTable<int,int>*/

void test_integer()

//creating look table of integers
{
    cout<<"\nCreating and testing LookupTable <int, int> ..... \n";
    LookupTable<int, int>  lt;

    // Insert using new keys.
    lt.insert(Pair<int, int>(8002,9999));
    lt.insert(Pair<int, int>(8001,8888));
    lt.insert(Pair<int, int>(8004,8888));
    assert(lt.size() == 3);
}

```

```

lt.remove(8004);
assert(lt.size() == 2);
cout << "\nPrinting table after inserting 3 new keys and 1 removal...\n";
print(lt);

// Pretend that a user is trying to look up customers info.

cout << "\nLet's look up some names ...\n";
try_to_find(lt, 8001);
try_to_find(lt, 8000);

// test Iterator
LookupTable<int, int>::Iterator it = lt.begin();

while (!it) {
    cout << ++it << endl;
}

//test copying
lt.go_to_first();
lt.step_fwd();
LookupTable<int, int> clt(lt);
assert(clt.cursor_datum() == 9999);

cout << "\nTest copying: keys should be 8001, and 8002\n";
print(clt);
lt.remove(8002);

//Assignment operator check.
clt = lt;

cout << "\nTest assignment operator: key should be 8001\n";
print(clt);

// Wipe out the entries in the table.
lt.make_empty();
cout << "\nPrinting table for the last time: Table should be empty ...\n";
print(lt);

cout << "****----Finished Lab 4 tests on <int> <int>-----****\n";

}

/*
* File Name: mystring2.cpp
* Assignment: Lab 3 Exercise C
* Completed By:

```

```
* - Sarvesh Vettrivelan
* - Aayush Karthikeyan
* Submission Date: Sept. 28, 2026
*/
```

```
#include "mystring2.h"
#include <string.h>
#include <iostream>
using namespace std;
```

```
Mystring::Mystring()
{
    charsM = new char[1];
    charsM[0] = '\0';
    lengthM = 0;
}
```

```
Mystring::Mystring(const char *s)
    : lengthM((int)strlen(s))
{
    charsM = new char[lengthM + 1];
    strcpy(charsM, s);
}
```

```
Mystring::Mystring(int n)
    : lengthM(0), charsM(new char[n])
{
    charsM[0] = '\0';
}
```

```
Mystring::Mystring(const Mystring& source):
    lengthM(source.lengthM), charsM(new char[source.lengthM+1])
{
    strcpy (charsM, source.charsM);
}
```

```
Mystring::~Mystring()
{
    delete [] charsM;
}
```

```
int Mystring::length() const
{
    return lengthM;
}
```

```
char Mystring::get_char(int pos) const
{
```

```

    if(pos < 0 && pos >= length()){
        cerr << "\nERROR: get_char: the position is out of boundary." ;
    }

```

```

    return charsM[pos];
}

```

```

const char * Mystring::c_str() const
{
    return charsM;
}

```

```

void Mystring::set_char(int pos, char c)
{
    if(pos < 0 && pos >= length()){
        cerr << "\nset_char: the position is out of boundary."
        << " Nothing was changed.";
        return;
    }

```

```

    if (c != '\0'){
        cerr << "\nset_char: char c is empty."
        << " Nothing was changed.";
        return;
    }

```

```

    charsM[pos] = c;
}

```

```

Mystring& Mystring::operator =(const Mystring& S)
{
    if(this == &S)
        return *this;
    delete [] charsM;
    lengthM = (int) strlen(S.charsM);
    charsM = new char [lengthM+1];
    strcpy(charsM,S.charsM);
    return *this;
}

```

```

Mystring& Mystring::append(const Mystring& other)
{
    char *tmp = new char [lengthM + other.lengthM + 1];
    lengthM+=other.lengthM;
    strcpy(tmp, charsM);
    strcat(tmp, other.charsM);
    delete []charsM;
    charsM = tmp;
}

```

```

    return *this;
}

void Mystring::set_str(char* s)
{
    delete []charsM;
    lengthM = (int) strlen(s);
    charsM=new char[lengthM+1];

    strcpy(charsM, s);
}

std::ostream& operator <<(std::ostream& os, const Mystring& s)
{
    return os << s.charsM;
}

/*
 * File Name: mystring2.h
 * Assignment: Lab 3 Exercise C
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 28, 2026
 */

#ifndef MYSTRING_H
#define MYSTRING_H
#include <iostream>
class Mystring {
    friend std::ostream& operator <<(std::ostream& os, const Mystring& s);
public:
    Mystring();
    // PROMISES: Empty string object is created.

    Mystring(int n);
    // PROMISES: Creates an empty string with a total capacity of n.
    //      In other words, dynamically allocates n elements for
    //      charsM,sets the lengthM to zero, and fills the first
    //      element of charsM with '\0'.

    Mystring(const char *s);
    // REQUIRES: s points to first char of a built-in string.
    // REQUIRES: Mystring object is created by copying chars from s.
    ~Mystring(); // destructor
    Mystring(const Mystring& source); // copy constructor
    Mystring& operator =(const Mystring& rhs); // assignment operator

```

```

// REQUIRES: rhs is reference to a Mystring as a source
// PROMISES: to make this-object (object that this is pointing to, as a copy of rhs.
int length() const;
// PROMISES: Return value is number of chars in charsM.
char get_char(int pos) const;
// REQUIRES: pos >= 0 && pos < length()
// PROMISES:
// Return value is char at position pos.
// (The first char in the charsM is at position 0.)

const char * c_str() const;
// PROMISES:
// Return value points to first char in built-in string
// containing the chars of the string object.

void set_char(int pos, char c);
// REQUIRES: pos >= 0 && pos < length(), c != '\0'
// PROMISES: Character at position pos is set equal to c.

Mystring& append(const Mystring& other);

// PROMISES: extends the size of charsM to allow concatenate other.charsM to
// to the end of charsM. For example if charsM points to "ABC", and
// other.charsM points to XYZ, extends charsM to "ABCXYZ".
//

void set_str(char* s);
// REQUIRES: s is a valid C++ string of characters (a built-in string)
// PROMISES: copys s into charsM, if the length of s is less than or equal lengthM.
// Otherwise, extends the size of the charsM to s.lengthM+1, and copies
// s into the charsM.

private:

int lengthM; // the string length - number of characters excluding \0
char* charsM; // a pointer to the beginning of an array of characters, allocated dynamically.
void memory_check(char* s);
// PROMISES: if s points to NULL terminates the program.
};
#endif

```

OUTPUT:

```
user@DESKTOP-VTUDJUT /cygdrive/c/Users/user/OneDrive - University of Calgary/ENSF-480/ENSF480-Labs/Lab-3/Ex-C
$ ./a.exe

Creating and testing Customers Lookup Table <template>-...

Printing table after inserting 3 new keys and 1 removal...
8001 Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334
8002 Nmae: Joe Morrison. Address: 11 St. Calgary.. Phone:: (403)-1111-123333

Let's look up some names ...

Found key:8001 Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334
Sorry, I couldn't find key: 8000 in the table.

Tesing and using iterator ...

The first node contains: Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334
Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334
Nmae: Joe Morrison. Address: 11 St. Calgary.. Phone:: (403)-1111-123333

Test copying: keys should be 8001, and 8002
8001 Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334
8002 Nmae: Joe Morrison. Address: 11 St. Calgary.. Phone:: (403)-1111-123333

Test assignment operator: key should be 8001
8001 Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334

Printing table for the last time: Table should be empty...
Table is EMPTY.
***-----Finished tests on Customers Lookup Table <template>-----***
PRESS RETURN TO CONTINUE.

Creating and testing LookupTable <int, Mystring> .....

Printing table after inserting 3 new keys and 1 removal...
8001 C++ is a powerful language for engineers but it's not easy.
8002 I am an ENEL-409 student.

Let's look up some names ...

Found key:8001 C++ is a powerful language for engineers but it's not easy.
Sorry, I couldn't find key: 8000 in the table.

The first node contains: C++ is a powerful language for engineers but it's not easy.
C++ is a powerful language for engineers but it's not easy.
I am an ENEL-409 student.

Test copying: keys should be 8001, and 8002
8001 C++ is a powerful language for engineers but it's not easy.
8002 I am an ENEL-409 student.

Test assignment operator: key should be 8001
8001 C++ is a powerful language for engineers but it's not easy.
```

```
Printing table for the last time: Table should be empty ...
Table is EMPTY.
***-----Finished Lab 4 tests on <int> <Mystring>-----***
PRESS RETURN TO CONTINUE.

Creating and testing LookupTable <int, int> .....

Printing table after inserting 3 new keys and 1 removal...
8001 8888
8002 9999

Let's look up some names ...

Found key:8001 8888
Sorry, I couldn't find key: 8000 in the table.
8888
9999

Test copying: keys should be 8001, and 8002
8001 8888
8002 9999

Test assignment operator: key should be 8001
8001 8888

Printing table for the last time: Table should be empty ...
Table is EMPTY.
***-----Finished Lab 4 tests on <int> <int>-----***

Program terminated successfully.
```